

ADR 集合

团队：第67组 日期：2026/5/2

ADR-001

ADR-001: 采用分层单体架构以优先保障10周内可交付的MVP

状态

待讨论

背景

CampusHub 是一个高校校园互助平台，学生可以发布任务（快递代取、拼单等）、接单、完成订单后双向互评。项目由4名软件架构经验尚浅的学生开发者组成的团队负责，总周期仅10周，必须交付具备注册登录、需求发布与浏览、接单、订单状态流转等 **Must Have** 功能的 MVP。非功能需求包括支撑500人同时在线、核心 API 响应时间低于2秒，保护学生隐私（不强制实名，支持匿名发布与评价），以及集成敏感词内容审核。**Should Have** 功能如表单的匿名选项、评价信用、基础论坛、站内通知等将在 MVP 交付后视进度考虑，而即时通讯、智能推荐、复杂信用模型等明确列为 **Won't Have**。在此约束下，团队需要从架构风格上做出根本性决策，以平衡10周内的交付可行性与未来的可维护性及可扩展性。核心的架构分歧点在于：是应选择开发上手的单体架构，还是追求长期可扩展性的微服务架构。

决策

我们决定采用 分层单体架构（**Layered Monolith**），以模块化的后端单体应用承载所有核心业务能力，前端使用 **Vue.js** 构建单页应用，整体通过 **Docker** 容器化部署。后端将严格按照表现层（API）、业务逻辑层（领域服务）和数据访问层（仓库）进行垂直分层，并按功能领域（用户、任务、订单、评价、论坛、通知等）进行水平模块划分。

理由

1. 最大化10周内交付的可行性，放弃微服务带来的分布式复杂性

在4人团队且经验尚浅的前提下，微服务架构需要掌握服务发现、负载均衡、分布式事务、跨服务通信（**REST/gRPC**）、合约测试、分布式日志与追踪等多重技能，学习曲线陡峭，会严重挤占功能开发时间。分层单体架构只需依赖 **Flask/Django** 内置的请求-响应机制和单一数据库，团队成员现有的 **Python + Vue + Docker** 经验即可覆盖全部开发与部署工作，从而将有限的时间聚焦在实现核心业务逻辑上。我们放弃了微服务带来的独立部署与独立演进的收益，换取了极短的开发启动耗时和确定的交付节奏。

2. 降低集成与运维的认知负荷，保障 MVP 稳定性

微服务架构意味着需要处理多服务间的部分失败、超时重试、最终一致性等棘手问题，还要搭建和维护 **CI/CD** 流水线、容器编排（如 **Docker Compose** 过渡到 **Kubernetes**）等基础设施。对于一个仅需支撑500并发、核心 API 响应 <2 秒的系统而言，这些复杂性在当前阶段属于过度设计。单体应用通过单进程部署和 **Docker** 容器化，即可用极低的运维成本满足性能目标，并能利用 **Gunicorn** 等 **WSGI** 服务器水平扩展进程数。我们放弃了微服务的细粒度弹性伸缩能力，但在 MVP 阶段几乎不会有这种需求，等到需要时再评估拆分远比提前引入分布式代价更低。

3. 通过严格的模块化设计为未来演进留出空间，避免向下牺牲长期可维护性

直接选择单体架构的风险在于日后可能成长为“大泥球”，难以维护和扩展。为弥补这一牺牲，我们在分层单体内强制执行功能模块边界：每个功能领域（如 **users**、**tasks**、**orders**、**reviews**、**forum**、**notifications**）拥有独立的服务、仓库和 API 资源，模块间通过定义好的接口（如 **Python** 的抽象基类或明确的函数签名）通信，严禁跨模块直接访问数据表。这种松耦合的模块化单体既可以维持快速的开发反馈循环，又使得未来若某模块（例如内容审核、站内通知）需要独立伸缩或重写时，可以相对平滑地抽离为独立的微服务，而不必重写整个系统。我们并没有放弃长期可扩展性，而是将其推迟到实际需要并拥有更多工程资源时再实现，当前以严格的架构约束作为折衷。

后果

- 正面影响：

1. 开发、构建、测试和部署流程极简，4人团队可在10周内集中精力交付完整的核心功能闭环。

2. 单一代码库和共享数据库降低了调试和问题定位的难度，利于经验尚浅的团队快速学习与协作。
3. **Docker** 化单体部署模式即可满足 MVP 的并发与延迟要求，无需额外的基础设施投资。
4. 预先定义的模块边界与接口规则从源头抑制耦合，即使不拆分，代码库也具备较高的可读性与可维护性。

- 负面影响：

1. 随着功能迭代，若模块间接口纪律松弛，代码库仍存在耦合蔓延、成为大泥球的潜在风险。
2. 未来若某一模块（如论坛或通知）负载显著增长，无法单独扩缩容，必须整体扩展整个应用实例，资源利用不够精细。
3. 持续交付的效率会随代码库膨胀而下降，全量构建和测试的时间将逐渐变长。

- 需要关注的风险：

1. 团队可能因进度压力而打破模块边界，需要从首日起建立代码审查、架构约束检查（如用 **linter** 或架构测试工具约束导入方向）等实践来持分与模块化的完整性。
2. 若 MVP 校园上线后用户量或功能需求爆发式增长，可能出现性能瓶颈或模块耦合导致的迭代速度骤降。缓解措施是持续监控核心 API 的响应时间与模块间的耦合度，将“可拆分性”作为非功能性指标纳入代码评审，一旦某些指标逼近阈值，优先将该模块剥离为独立服务。
3. **Django** 的 “apps” 或 **Flask** 蓝图天然支持模块化划分，应充分利用这些机制，将每个功能领域实现为可独立替换的组件，降低未来迁移成本。

人工修订后ADR

用 `~~删除~~` 和 `**新增**` 标记出修订差异的版本。

ADR-001：采用分层单体架构以优先保障**10**周内可交付的**MVP**

状态

待讨论

背景

CampusHub 是一个高校校园互助平台~~，学生可以发布任务、接单、完成订单后互评。~~ ****学生可发布任务（快递代取、拼单等）、接单、完成订单后双向互评。**** 项目由4名软件架构经验尚浅的学生开发者组成的团队负责，总周期仅**10**周，必须交付具备注册登录、需求发布与浏览、接单、订单状态流转等 **Must Have** 功能的 MVP。****非功能需求包括支撑500人同时在线、核心 API 响应时间低于2秒，保护学生隐私（不强制实名，支持匿名发布与评价），以及集成敏感词内容审核。**** **Should Have** 功能如表单的匿名选项、评价信用、基础论坛、站内通知等将在 MVP 交付后视进度考虑，而即时通讯、智能推荐、复杂信用模型等明确列为 **Won't Have**。

在此约束下，团队需要从架构风格上做出根本性决策，以平衡**10**周内的交付可行性与未来的可维护性及可扩展性。核心的架构分歧点在于：是应选择开发上手的单体架构，还是追求长期可扩展性的微服务架构。

决策

我们决定采用 ****分层单体架构（Layered Monolith）****，~~后端使用 Python（Flask/Django）构建单体应用，前端使用 Vue.js~~ ****以模块化的后端单体应用承载所有核心业务能力，前端使用 Vue.js 构建单页应用，整体通过 Docker 容器化部署。后端将严格按照表现层（API）、业务逻辑层（领域服务）和数据访问层（仓库）进行垂直分层，并按功能领域（用户、任务、订单、评价、论坛、通知等）进行水平模块划分。****

理由

- ~~1. 开发简单，团队熟悉 Python 和 Vue，能快速启动。~~
- ~~2. 单体应用部署容易，不需要服务发现等复杂基础设施。~~
- ~~3. 未来可以拆分为微服务。~~

****1. 最大化10周内交付的可行性，放弃微服务带来的分布式复杂性****

在4人团队且经验尚浅的前提下，微服务架构需要掌握服务发现、负载均衡、分布式事务、跨服务通信（REST/gRPC）、合约测试、分布式日志与追踪等多重技能，学习曲线陡峭，会严重挤占功能开发时间。分层单体架构只需依赖 **Flask/Django** 内置的请求-响应机制和单一数据库，团队成员现有的

Python + Vue + Docker 经验即可覆盖全部开发与部署工作，从而将有限的时间聚焦在实现核心业务逻辑上。我们放弃了微服务带来的独立部署与独立演进的收益，换取了极短的开发启动耗时和确定的交付节奏。

****2. 降低集成与运维的认知负荷，保障 MVP 稳定性****

微服务架构意味着需要处理多服务间的部分失败、超时重试、最终一致性等棘手问题，还要搭建和维护 CI/CD 流水线、容器编排（如 Docker Compose 过渡到 Kubernetes）等基础设施。对于一个仅需支撑500并发、核心 API 响应 <2 秒的系统而言，这些复杂性在当前阶段属于过度设计。单体应用通过单进程部署和 Docker 容器化，即可用极低的运维成本满足性能目标，并能利用 Gunicorn 等 WSGI 服务器水平扩展进程数。我们放弃了微服务的细粒度弹性伸缩能力，但在 MVP 阶段几乎不会有这种需求，等到需要时再评估拆分远比提前引入分布式代价更低。

****3. 通过严格的模块化设计为未来演进留出空间，避免向下牺牲长期可维护性****

直接选择单体架构的风险在于日后可能成长为“大泥球”，难以维护和扩展。为弥补这一牺牲，我们在分层单体内强制执行功能模块边界：每个功能领域（如 users、tasks、orders、reviews、forum、notifications）拥有独立的服务、仓库和 API 资源，模块间通过定义好的接口（如 Python 的抽象基类或明确的函数签名）通信，严禁跨模块直接访问数据表。这种松耦合的模块化单体既可以维持快速的开发反馈循环，又使得未来若某模块（例如内容审核、站内通知）需要独立伸缩或重写时，可以相对平滑地抽离为独立的微服务，而不必重写整个系统。我们并没有放弃长期可扩展性，而是将其推迟到实际需要并拥有更多工程资源时再实现，当前以严格的架构约束作为折衷。**

后果

- 正面影响：~~开发速度快，部署简单。~~

****1. 开发、构建、测试和部署流程极简，4人团队可在10周内集中精力交付完整的核心功能闭环。****

****2. 单一代码库和共享数据库降低了调试和问题定位的难度，利于经验尚浅的团队快速学习与协作。****

****3. Docker 化单体部署模式即可满足 MVP 的并发与延迟要求，无需额外的基础设施投资。****

****4. 预先定义的模块边界与接口规则从源头抑制耦合，即使不拆分，代码库也具备较高的可读性与可维护性。****

- 负面影响：~~可能变成大泥球。~~

****1. 随着功能迭代，若模块间接口纪律松弛，代码库仍存在耦合蔓延、成为大泥球的潜在风险。****

****2. 未来若某一模块（如论坛或通知）负载显著增长，无法单独扩缩容，必须整体扩展整个应用实例，资源利用不够精细。****

****3. 持续交付的效率会随代码库膨胀而下降，全量构建和测试的时间将逐渐变长。****

- 需要关注的风险：~~注意保持模块边界。~~

****1. 团队可能因进度压力而打破模块边界，需要从首日起建立代码审查、架构约束检查（如用 linter 或架构测试工具约束导入方向）等实践来维持分层与模块化的完整性。****

****2. 若 MVP 校园上线后用户量或功能需求爆发式增长，可能出现性能瓶颈或模块耦合导致的迭代速度骤降。缓解措施是持续监控核心 API 的响应时间与模块间的耦合度，将“可拆分性”作为非功能性指标纳入代码评审，一旦某些指标逼近阈值，优先将该模块剥离为独立服务。****

****3. Django 的“apps”或 Flask 蓝图天然支持模块化划分，应充分利用这些机制，将每个功能领域实现为可独立替换的组件，降低未来迁移成本。****

AI 辅助记录

- AI 初稿内容摘要：

建议采用分层单体架构，理由为开发简单、团队熟悉、可快速上线，未来可拆分为微服务。未具体分析10周约束与质量属性之间的权衡，也未给出模块化实施的严格方案。

- 人工修订内容：

1. 补充了500并发、敏感词审核、匿名等非功能约束的背景描述。

2. 将三条笼统理由替换为基于“10周交付可行性vs.长期可维护性”的详细权衡分析。
3. 明确了通过水平模块划分和接口纪律来补偿单体牺牲的可扩展性。
4. 详细列出了正面/负面影响及具体风险缓解措施。

- 修订理由:

AI 初稿未体现该项目关键的质量属性权衡（交付速度 vs 可扩展性），只停留在常识性优点罗列，未给出可执行的模块化约束和风险应对策略。修订使其成为一条有记录价值的架构决策，真正指导团队在压缩周期内的设计纪律。

ADR-002

ADR-002: 采用进程内轻量事件驱动处理非核心流程

状态

已接受

背景

我负责CampusHub校园互助平台的架构设计，结合项目约束（4人学生团队、10周交付MVP、500人在线、核心接口响应<2秒），平台在订单状态变更、评价完成、论坛发帖等场景下，需触发站内通知、信用分更新等非核心流程。若同步处理会拉长核心接口响应时间，影响用户体验；若引入外部消息队列（RabbitMQ/Kafka），会提升架构复杂度，超出团队能力且影响交付周期，需在性能、解耦、复杂度之间权衡。

决策

我决定采用后端框架（Python Flask）内置的进程内轻量异步事件机制，处理站内通知、信用分更新等非核心流程，不引入任何外部消息中间件，确保架构极简、可落地。

理由

- 适配项目约束：贴合10周交付目标，无需部署、配置外部消息队列，降低学习与运维成本，适配4人学生团队能力，确保按时完成架构落地。
- 保障核心体验：异步处理非核心流程，不阻塞发单、接单、确认完成等核心接口，配合Redis缓存，可确保核心接口响应时间<2秒，满足性能要求。
- 实现业务解耦：通知、信用分更新模块不侵入核心业务逻辑，符合单一职责原则，便于后期维护与迭代，降低调试难度。
- 适配业务规模：校园平台峰值 500 人在线，事件量较小，进程内异步完全可支撑，无需复杂的事件持久化处理，简化开发流程。

后果

- 正面影响
 - 核心接口响应速度提升，保障用户使用体验，满足项目性能要求。
 - 架构复杂度低，开发、调试效率高，契合学生团队能力，确保按时交付。
 - 业务模块解耦，后期维护、功能迭代更便捷，降低出错概率。
- 负面影响
 - 事件无持久化，应用重启可能丢失少量非核心通知（不影响核心业务）。
 - 不支持跨实例广播，但若未来扩展为分布式系统，需调整异步机制。
- 风险控制
 - 严格区分核心与非核心流程，仅将通知、信用分更新等非核心操作纳入异步处理，核心业务保持同步执行。
 - 规范异步事件编码规范，避免异步逻辑过多导致调试困难，做好日志记录便于问题定位。

AI 辅助记录

- AI 初稿摘要：建议引入RabbitMQ消息队列实现异步解耦，强调高可用性，未考虑学生团队能力与10周交付周期，属于过度设计。
- 人工修订：放弃外部消息队列，改为进程内轻量异步事件机制，明确适用场景与风险

控制，补充团队与周期适配性理由。

- 修订理由：AI未结合项目实际约束（学生团队、短周期），盲目推荐重型方案，我结合架构设计工作实际，修正为轻量化可落地方案，确保决策务实、可交付。

ADR-003

ADR-003: 采用按业务模块划分的单体分层架构

状态

已接受

背景

CampusHub 需要在 10 周内完成一个可运行、可演示、可维护的校园互助平台。团队规模为 4 人，当前

技术基础为 Vue 前端与 Spring Boot 后端。系统包含用户、需求、订单、评价、通知、管理等功能，需要

在保证开发效率的同时维持清晰的职责边界。

决策

我们决定采用“前后端分离 + 后端单体分层 + 按业务模块组织代码”的架构方式。

理由

- 相比微服务，单体分层架构开发成本更低，更适合课程项目周期。
- 按业务模块组织代码，能够在不增加部署复杂度的前提下保持结构清晰。
- 该方案兼顾当前交付效率与后续一定程度的可扩展性。
- 团队成员对 Vue、Spring Boot、RESTful API 的学习和使用成本较低。

后果

- 正面影响：

开发与联调成本较低。

代码结构清晰，便于分工协作。

部署简单，适合课程项目推进。

- 负面影响：

当业务继续膨胀时，单体应用可能出现模块耦合上升的问题。

后续扩展性不如天然拆分的微服务架构。

- 风险控制：

如果模块边界定义不清，容易在 Service 层出现职责混杂。

需要通过包结构、接口定义和代码规范控制耦合。

AI 辅助记录

- AI 初稿内容摘要

AI 建议在单体分层架构和微服务架构之间进行选择。

AI 指出微服务更有扩展性，而单体分层更适合小团队短周期项目。

- 人工修订内容

将“单体分层架构”进一步明确为“按业务模块划分的单体分层架构”。

补充强调模块边界与代码组织方式，而不仅仅停留在技术框架层面。

- 修订理由

AI 给出的建议较为概括，缺少对“如何在单体中保持模块清晰”的具体说明。

经过人工补充后，决策内容更符合当前项目的实际落地需要